21-docker

Modul 21: Docker — Kontainer untuk Aplikasi Modern

Target: Siswa SMK RPL
Tujuan: Pahami Docker, bikin Dockerfile & docker-compose, deploy aplikasi Node.js/TypeScript full-stack.

---

1. Apa Itu Docker?

Docker adalah platform untuk **mengemas aplikasi + dependensi** ke dalam **kontainer**. Kontainer = lingkungan terisolasi yang bisa jalan di mana aja.

Kontainer vs VM (Virtual Machine)

Aspek	Kontainer (Docker)	Virtual Machine
OS	Pakai kernel host	Punya OS tamu sendiri
Ukuran	MB	GB
Boot	Detik	Menit
Isolasi	Proses-level	Hardware-level
Resource	Ringan	Berat (butuh RAM/CPU dedicated)

Analogi:

- VM = rumah penuh — setiap rumah punya fondasi, listrik, pipa sendiri.
- Kontainer = apartemen — satu gedung, satu fondasi, tiap unit terpisah tapi pakai infrastruktur bareng.

---

2. Dockerfile — Resep Image

`Dockerfile` adalah file teks berisi instruksi untuk membangun **image** Docker. Image = template read-only untuk membuat kontainer.

Instruksi Dasar

```dockerfile
# FROM — base image (wajib)
FROM node:20-alpine

# WORKDIR — set direktori kerja di dalam kontainer
WORKDIR /app

# COPY — salin file dari host ke kontainer
COPY package*.json ./
COPY src/ ./src
COPY public/ ./public

# RUN — eksekusi perintah saat build
RUN npm ci --omit=dev

# EXPOSE — info port yang dibuka (dokumentasi, bukan buka port)
EXPOSE 3000

# CMD — perintah jalan saat kontainer start (hanya satu CMD)
CMD ["node", "dist/index.js"]
```

Contoh Dockerfile Sederhana — Express.js

```dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Contoh Dockerfile — React (Nginx)

```dockerfile
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
```

```
RUN npm run build

# Stage 2: Serve pakai Nginx
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

---

3. Multi-stage Build — Kecilkan Ukuran Image

Multi-stage = pakai **banyak FROM** dalam satu Dockerfile. Stage awal buat build, stage akhir cuma ambil hasilnya. Image akhir jadi kecil (tanpa dev tools, node_modules, dll).

Kenapa Penting?

- Image `node:20-slim` ~200 MB
- Image `node:20-alpine` ~120 MB
- Image produksi setelah multi-stage bisa < 50 MB
- Transfer lebih cepat, storage hemat, security lebih baik

Contoh — Express TypeScript

```
# === Stage 1: Build & Compile ===
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json tsconfig*.json ./
RUN npm ci
COPY src/ ./src
RUN npm run build

# === Stage 2: Production ===
FROM node:20-alpine AS production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --omit=dev --ignore-scripts
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Latihan: cek ukuran image dengan `docker images`. Bandingkan sebelum dan sesudah multi-stage.

---

4. docker-compose.yml — Orkestrasi Multi-Kontainer

Pakai `docker-compose.yml` kalau aplikasi butuh lebih dari satu service (e.g. backend + database + redis).

Struktur Dasar

```yaml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:
      - db
      - redis

  db:
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: myapp
      POSTGRES_PASSWORD: secret

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

Penjelasan Key

Key	Fungsi
`services`	Daftar kontainer yang dijalankan
`build`	Path ke Dockerfile
`image`	Pakai image siap pakai dari registry
`ports`	`"host:container"` — forwarding port
`volumes`	Persist data (host volume atau named volume)

Key	Fungsi
`environment` / `env_file`	Variabel lingkungan
`depends_on`	Urutan start (tunggu service lain)

Contoh Full-stack — FE (React) + BE (Express) + DB (PostgreSQL) + Redis

```yaml
version: '3.8'

services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "80:80"
    depends_on:
      - backend
    environment:
      VITE_API_URL: http://localhost:3000

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    env_file: ./backend/.env
    depends_on:
      - db
      - redis
    volumes:
      - ./backend/uploads:/app/uploads

  db:
    image: postgres:16-alpine
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: p4ssw0rd
      POSTGRES_DB: app_db
```

```
  redis:
    image: redis:7-alpine
    restart: always
    ports:
      - "6379:6379"
    command: redis-server --requirepass redispass

volumes:
  postgres_data:
```

Jalankan dengan:

```
docker compose up -d
docker compose logs -f
docker compose down
```

---

5. Docker Networking

Bridge (default)

Setiap kontainer dapat IP internal di jaringan `bridge`. Bisa saling akses via IP, tapi lebih baik pakai service name (docker-compose).

```
docker network ls
docker inspect bridge
```

Host

Kontainer pakai network host langsung — port terbuka tanpa NAT. Cepat, tapi kurang isolasi.

```
services:
  app:
    network_mode: host
```

Custom Network

Buat jaringan terisolasi sendiri. Service name = hostname.

```
docker network create my-network
docker run --network my-network --name api my-api
```

Di docker-compose:

```
services:
  backend:
    networks:
```

```
      - app-net
  db:
    networks:
      - app-net

networks:
  app-net:
    driver: bridge
```

---

6. Volumes vs Bind Mounts

Keduanya untuk **persist data** — data tetap ada meski kontainer dihapus.

	Volume	Bind Mount
Lokasi	`docker volume inspect`	Path host (user-defined)
Manajemen	Docker yang urus	Manual (path host)
Backup	`docker run --rm -v volume:/data ...`	Langsung copy dari folder host
Kinerja	Lebih cepat	Tergantung OS
Cocok	Data DB, file aplikasi	Development, hot-reload

```
# Volume
docker volume create my-data
docker run -v my-data:/app/data my-image

# Bind Mount — cocok dev (file sinkron)
docker run -v $(pwd):/app my-image
```

Di docker-compose:

```
services:
  app:
    volumes:
      - ./src:/app/src          # bind mount — hot reload
      - ./uploads:/app/uploads  # bind mount
      - node_modules:/app/node_modules  # named volume

volumes:
  node_modules:
```

---

7. Docker Hub & Private Registry

Docker Hub

Registry publik. Pull image gratis, push image perlu akun.

```
# Login
docker login

# Push image
docker build -t username/nama-image:tag .
docker push username/nama-image:tag
```

Private Registry

Untuk internal team. Bisa self-hosted atau pakai layanan cloud (GitHub Packages, AWS ECR, GitLab Registry).

```
# Self-hosted
docker run -d -p 5000:5000 --name registry registry:2
docker tag my-image localhost:5000/my-image
docker push localhost:5000/my-image
```

Gambar alur: `Build → Tag → Push → Pull → Run`.

---

8. Docker untuk Node.js / TypeScript

Express — Dockerfile Produksi

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json tsconfig*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS prod
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --omit=dev
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Best Practices

1. **Gunakan `.dockerignore`** — hindari salin `node_modules`, `.git`, `.env`
2. **`npm ci` bukan `npm install`** — install sesuai lockfile, lebih cepat & reproducible
3. **Pisahkan COPY** — `COPY package*.json ./` dulu, baru `COPY . .` — manfaatkan cache layer
4. **Gunakan `--omit=dev`** — dev dependencies tidak masuk image produksi
5. **Jalankan sebagai non-root** — security

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
```

Contoh `.dockerignore`

```
node_modules
dist
.git
.env
*.md
.DS_Store
coverage
```

---

9. Cheatsheet Perintah Docker

Image

```
docker build -t nama-image:tag .          # Build image
docker images                              # Daftar image
docker rmi nama-image:tag                  # Hapus image
docker pull node:20-alpine                 # Download image dari registry
docker push user/nama-image:tag            # Upload ke registry
```

Kontainer

```
docker run -d --name myapp -p 3000:3000 my-image   # Jalankan container
docker ps                                           # Daftar container jalan
docker ps -a                                        # Semua container
docker stop myapp                                   # Stop
docker start myapp                                  # Start ulang
docker rm myapp                                     # Hapus
docker logs -f myapp                                # Lihat log (follow)
```

```
docker exec -it myapp sh                                    # Masuk shell container
docker cp file.txt myapp:/app/                              # Copy file ke container
```

Docker Compose

```
docker compose up -d          # Build & jalankan semua service
docker compose down           # Stop & hapus container + network
docker compose down -v        # Juga hapus volume
docker compose logs -f        # Stream log semua service
docker compose ps             # Status service
docker compose build          # Build ulang image tanpa start
docker compose restart        # Restart semua service
```

Volume & Network

```
docker volume create my-vol    # Buat volume
docker volume ls               # Daftar volume
docker network create net      # Buat network
docker network ls              # Daftar network
docker inspect myapp           # Info detail container (IP, mount, dll)
```

Cleanup

```
docker system prune -a         # Hapus image/container/network tak terpakai
docker container prune         # Hapus container stop
docker image prune             # Hapus dangling image
```

---

10. Ringkasan

- **Docker** → kemas aplikasi + dependensi jadi kontainer ringan.
- **Dockerfile** → resep build image (FROM, WORKDIR, COPY, RUN, CMD, EXPOSE).
- **Multi-stage** → kecilkan ukuran image produksi.
- **docker-compose.yml** → orkestrasi multi-service (FE + BE + DB + Redis).
- **Networking** → bridge (default), host (cepat), custom (isolasi).
- **Volumes** vs **Bind Mounts** → persist data; volume cocok produksi, bind mount cocok dev.
- **Docker Hub / Private Registry** → simpan & distribusi image.
- **Best Practice Node.js** → multi-stage, `.dockerignore`, non-root, layer cache.

Latihan Mandiri

1. Dockerfile-kan Express app sederhana. Build & run.
2. Tambah multi-stage. Bandingkan ukuran image.
3. Buat docker-compose dengan Express + PostgreSQL + Redis.
4. Push image ke Docker Hub.
5. Coba bind mount untuk hot-reload saat development.

”Works on my machine” — masalah selesai dengan Docker.